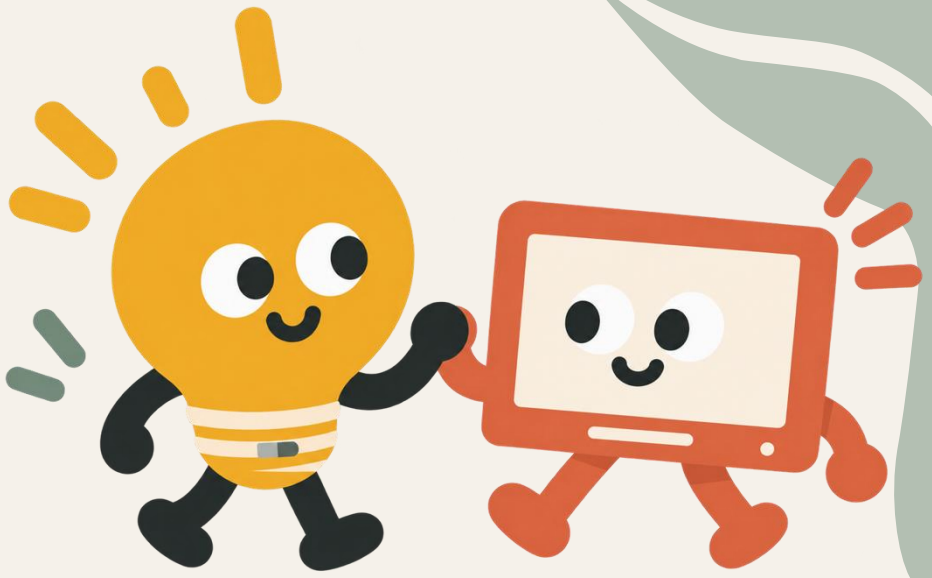
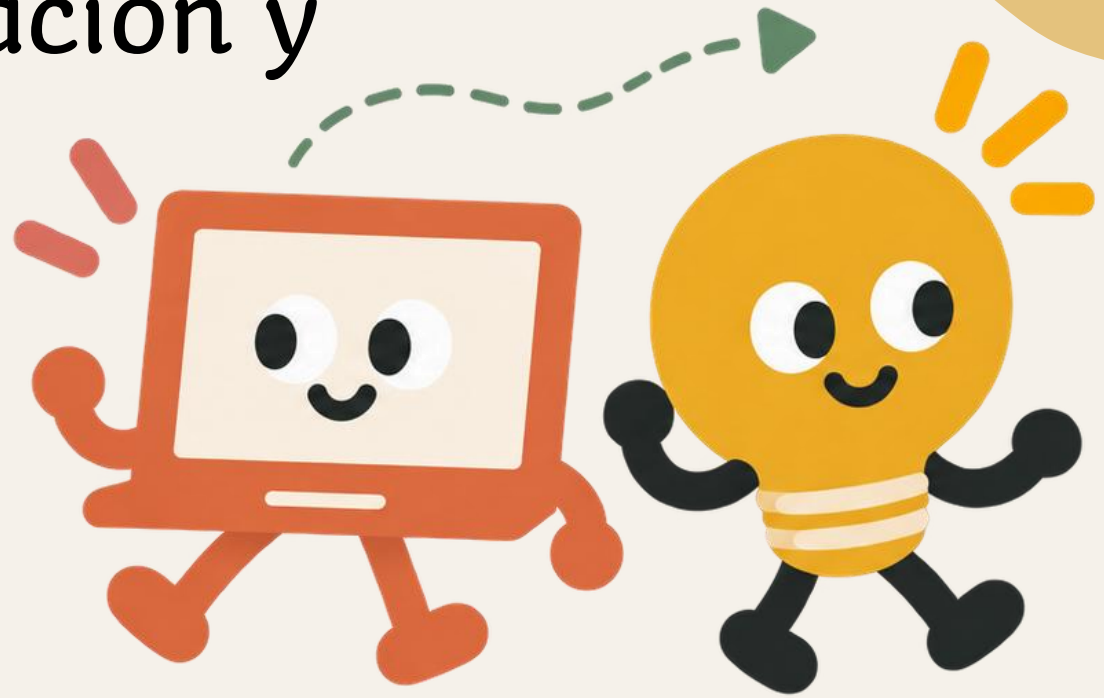
Módulo 6

Taller teórico-práctico
Del dataset a U-Net: segmentación de imágenes con Deep Learning

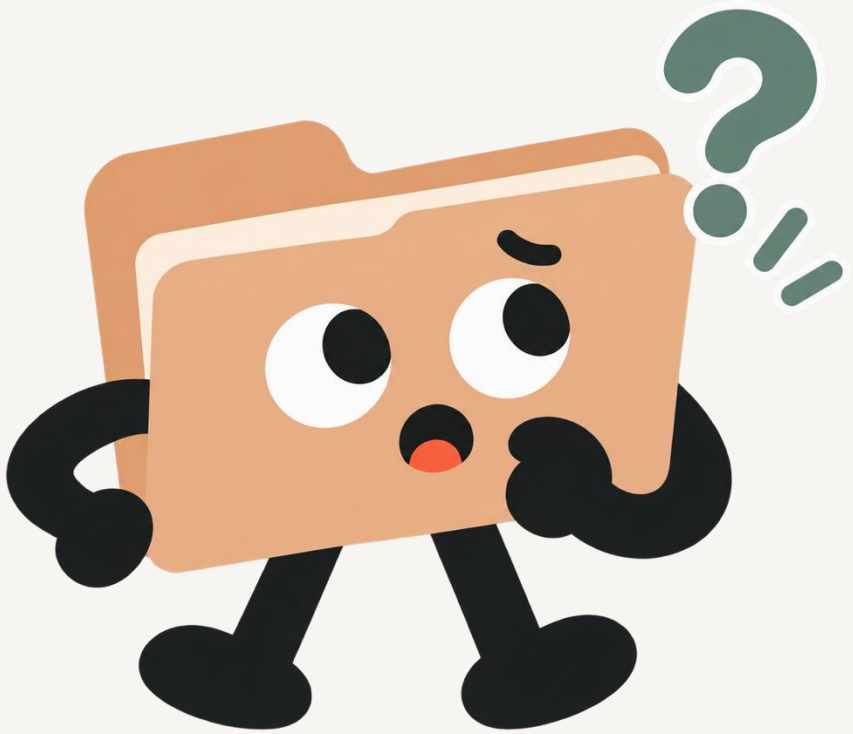
→ Entrenamiento, evaluación y análisis



Entrenamiento, evaluación y análisis de resultados



¿Podemos afirmar que el modelo segmentará correctamente imágenes nuevas únicamente porque su pérdida de entrenamiento disminuye?



Entrenamiento y validación

Usan la misma red y función de pérdida, pero con objetivos distintos: aprender los parámetros vs. estimar su desempeño.



Aspecto	Entrenamiento	Validación
Datos	`train_loader`	`val_loader`
Forward pass	✓	✓
Cálculo de pérdida	✓	✓
Backpropagation	✓	✗
Actualización de pesos	✓	✗
Modo del modelo	`model.train()`	`model.eval()`
Propósito	Aprender parámetros.	Estimar desempeño en datos no usados.

Puntos clave

- La validación no modifica los pesos.
- Sirve para tomar decisiones (p. ej., elegir un checkpoint o ajustar hiperparámetros).
- No sustituye al conjunto de prueba final.
- Usa las mismas transformaciones de tamaño y normalización que el entrenamiento, pero sin aumentos aleatorios.

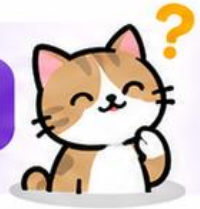
Validation loop (PyTorch)

```python
model.eval()                                  # 1. Modo evaluación
val_loss_sum = 0.0
val_samples = 0
with torch.no_grad():                         # 2. Sin gradientes
    for images, masks in los_loader:
        images = images.to(device)
        masks = masks.to(device)
        logits = model(images)                # 3. Forward pass
        loss = criterion(logits, masks)       # 4. Pérdida
        batch_size = images.size(0)
        val_loss_sum += loss.item() * batch_size  # 5. Acumula
        val_samples += batch_size

val_loss = val_loss_sum / val_samples         # 6. Media global
print(f"Validation loss: {val_loss:.4f}")
```

`model.eval()` ajusta capas como Dropout y BatchNorm al modo de evaluación, y `torch.no_grad()` evita construir el grafo de gradientes (menor uso de memoria).

Actividad



De las siguientes instrucciones, ¿cuáles pertenecen al validation loop?

`loss.backward()` `model.eval()` `optimizer.step()` `criterion(...)`

Curvas de aprendizaje: ¿está aprendiendo bien mi modelo?

Nos ayudan a entender cómo evoluciona el modelo durante el entrenamiento y si generaliza adecuadamente.

1 Misma función de pérdida

Evaluamos el mismo objetivo en entrenamiento y validación (p. ej., Dice Loss).



- **Training loss:** error en los datos con los que aprende.
- **Validation loss:** error en datos no usados para actualizar los pesos.

2 ¿Qué son las curvas de aprendizaje?



- Muestran cómo cambia la pérdida a lo largo de las épocas.
- Generalmente ambas disminuyen si el modelo aprende.

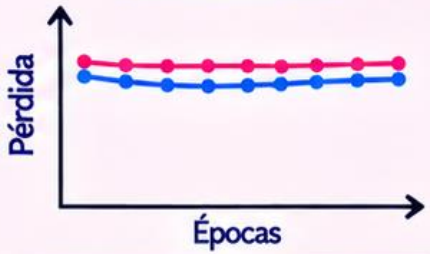
3 Generalización

Es la capacidad del modelo para realizar buenas predicciones en imágenes nuevas.



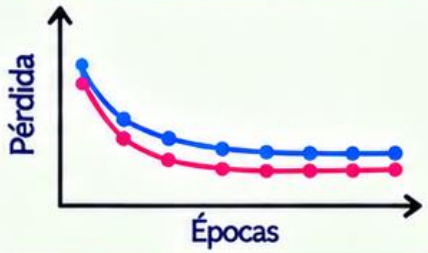
4 Comportamientos comunes

Subajuste



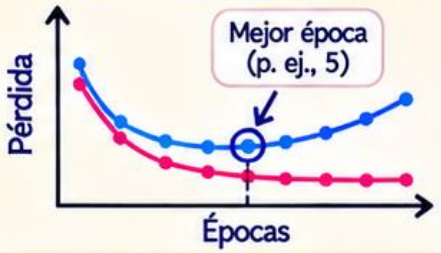
Ambas pérdidas son altas. El modelo aún no aprende suficientemente.

Aprendizaje favorable



Ambas pérdidas disminuyen y se estabilizan. El modelo aprende y generaliza bien.

Sobreajuste



La pérdida de entrenamiento sigue bajando, pero la de validación aumenta. El modelo se adapta demasiado a los datos de entrenamiento.

5 ¿Qué hacer?

Situación	Posibles acciones
Subajuste	• Entrenar más épocas. • Revisar la arquitectura. • Ajustar el learning rate.
Sobreajuste	• Aplicar aumentos de datos. • Usar regularización (p. ej., weight decay, dropout). • Detener el entrenamiento antes (early stopping).
Curvas inestables	• Revisar el learning rate. • Ajustar el batch size. • Verificar la calidad de los datos.

Idea clave



- No basta con que la pérdida de entrenamiento disminuya.
- También debemos analizar la **pérdida de validación** para saber si el modelo aprende y generaliza correctamente.

Las curvas son indicios, no diagnósticos definitivos.

Checkpoints: guardar y recuperar modelos

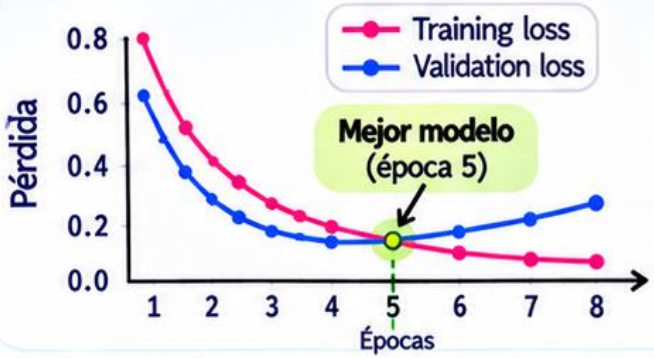
Permiten conservar el estado de un modelo y usar la mejor versión o continuar el entrenamiento.

¿Qué es un checkpoint?

Un archivo que almacena el estado del modelo en un momento del entrenamiento (p. ej., sus pesos).



¿Por qué no siempre la última época?



La menor pérdida de validación ocurre en la época 5, aunque la de entrenamiento sigue disminuyendo hasta la época 8.

Tipos de checkpoints

Último modelo	Pesos de la época más reciente.
Mejor modelo	Pesos del mejor resultado de validación.
Para reanudar	Pesos, estado del optimizador, época y otros datos.

Guardar el mejor modelo

```
best_val_loss = float("inf")

# Dentro del ciclo de épocas:
if val_loss < best_val_loss:
    best_val_loss = val_loss
    torch.save(
        model.state_dict(),
        "best_model.pth"
    )
```



Recuperar para inferencia

```
best_model = MiniUNet().to(device)
state_dict = torch.load(
  "best_model.pth",
   map_location=device,
   weights_only=True)
best_model.load_state_dict(state_dict)
best_model.eval()
```

Para salida binaria (1 canal):

```
prob = torch.sigmoid(logits)
```

Para 2 clases (2 canales):

```
prob = torch.softmax(logits, dim=1)
```

Guardar y recuperar un checkpoint completo (reanudar entrenamiento)

Guardar

```
torch.save({
  "epoch": epoch,
  "model_state_dict": model.state_dict(),
  "optimizer_state_dict": optimizer.state_dict(),
  "best_val_loss": best_val_loss
}, "checkpoint.pth")
```

Recuperar

```
checkpoint = torch.load(
  "checkpoint.pth",
   map_location=device,
   weights_only=True)
model.load_state_dict(
   checkpoint["model_state_dict"])
optimizer.load_state_dict(
   checkpoint["optimizer_state_dict"])
start_epoch = checkpoint["epoch"] + 1
best_val_loss = checkpoint["best_val_loss"]
```

En síntesis



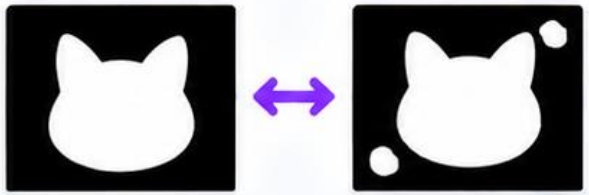
- **Entrena y valida:** evalúa el mismo objetivo con la misma función de pérdida.
- **Guarda el mejor modelo:** conserva la versión con menor pérdida de validación.
- **Elige el checkpoint adecuado:** solo los pesos para inferencia, o pesos + optimizador + época para reanudar.

Matriz de confusión en segmentación binaria

Evalúa, píxel por píxel, qué tan bien el modelo clasifica la mascota (1) y el fondo (0).

Comparación píxel a píxel

Máscara real (ground truth) ↔ Máscara predicha (modelo)



Significado de los elementos

TP	✓	**Verdadero positivo:** mascota → mascota (acierto).
FP	✗	**Falso positivo:** fondo → mascota (error).
TN	✓	**Verdadero negativo:** fondo → fondo (acierto).
FN	✗	**Falso negativo:** mascota → fondo (error).

Matriz de confusión (binaria)

Real \ Predicción	Mascota (1)	Fondo (0)
Mascota (1)	TP	FN
Fondo (0)	FP	TN

Ejemplo: 100 píxeles



- 40 píxeles: mascota
- 60 píxeles: fondo

Real \ Predicción	Mascota (1)	Fondo (0)
Mascota (1)	35 (TP)	5 (FN)
Fondo (0)	10 (FP)	50 (TN)

Métricas de segmentación

$$\text{Dice} = \frac{2TP}{2TP + FP + FN}$$

$$\text{Dice} \approx 0.824 \qquad \frac{70}{70 + 10 + 5}$$

$$\text{IoU} = \frac{TP}{TP + FP + FN}$$

$$\text{IoU} = 0.70 \qquad \frac{35}{35 + 10 + 5}$$

Nota: Dice e IoU no incluyen TN. Así, no se ven infladas por la gran cantidad de píxeles de fondo.

¿Por qué es importante?

Permite distinguir qué tipo de errores comete el modelo y cuantificar su desempeño en la segmentación.

Idea clave



La **matriz de confusión** resume los aciertos (TP, TN) y errores (FP, FN) y nos permite calcular métricas como Dice e IoU para evaluar la segmentación.

Métricas de evaluación en segmentación binaria

Se calculan a partir de TP, TN, FP y FN, y describen distintos aspectos del desempeño del modelo.

Matriz de confusión (ejemplo)



Real \ Predicción	Mascota (1)	Fondo (0)
Mascota (1)	TP = 35	FN = 5
Fondo (0)	FP = 10	TN = 50

Total: 100 píxeles

Resultados del ejemplo

Precision	0.778 (77.8 %)
Recall	0.875 (87.5 %)
F1 / Dice	0.824 (82.4 %)
Accuracy	0.850 (85.0 %)
IoU	0.700 (70.0 %)
Specificity	0.833 (83.3 %)

Métricas principales

Precision — $\frac{TP}{TP + FP}$ — De los píxeles que predice como mascota, ¿cuántos realmente lo son?

Recall (sensibilidad) — $\frac{TP}{TP + FN}$ — De los píxeles de la mascota real, ¿cuántos detecta el modelo?

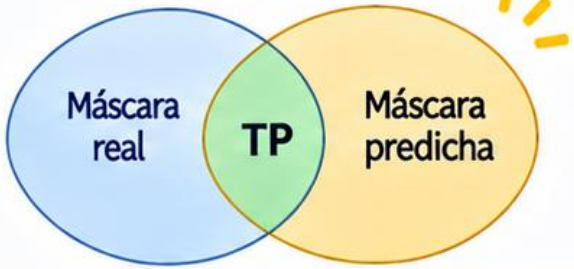
F1 / Dice — $\frac{2TP}{2TP + FP + FN}$ — Equilibrio entre Precision y Recall (coincidencia de la máscara).

Accuracy — $\frac{TP + TN}{TP + TN + FP + FN}$ — Proporción de todos los píxeles clasificados correctamente.

IoU (Intersection over Union) — $\frac{TP}{TP + FP + FN}$ — Proporción de la intersección respecto a la unión.

Specificity (especificidad) — $\frac{TN}{TN + FP}$ — De los píxeles de fondo real, ¿cuántos identifica correctamente?

IoU: intuición geométrica



Unión: TP + FP + FN

$$IoU = \frac{TP}{TP + FP + FN}$$

Puntos clave

- **Precision:** pocos falsos positivos (menos fondo como mascota).
- **Recall:** detecta la mayor parte de la mascota (menos falsos negativos).
- **Accuracy** puede ser engañosa si hay muchas más píxeles de fondo.



Para nuestro taller, nos enfocaremos principalmente en **Dice / F1**, **IoU**, **Precision y Recall**, y las complementaremos con otras métricas y la revisión visual de las máscaras.

Idea clave

No existe ua métrica universalmente mejor: cada una describe un aspecto diferente del resultado en segmentación.